Restaurant Sales Dashboard Project Report

Restaurant Sales Dashboard

Project Report

Student Name: Neha Gahlot

Roll Number: Not provided

Institution: Rungta International Skills University

Department: Bachelor of Computer Application Engineering with AI and ML, in association with Google

Submission Date: 24 October 2026

Abstract

The Restaurant Sales Dashboard is a Python-based data analysis project designed to process restaurant sales records, calculate important revenue metrics, and present the results through graphical visualizations. The project uses a CSV dataset containing sales dates, menu items, food categories, quantities sold, and unit prices.

The system uses pandas for data loading, cleaning, grouping, and numerical analysis. Matplotlib is used to generate charts illustrating revenue by menu item and daily revenue trends. Automated tests implemented with pytest verify important calculations and data-cleaning behavior.

The included demonstration dataset contains six records across two dates. Analysis of these records produces a total calculated revenue of ₹15,040. The project demonstrates a basic data analytics workflow, from loading raw data to cleaning, analysis, visualization, and testing.

Keywords: Python, Restaurant Sales, Data Analysis, pandas, Matplotlib, Data Cleaning, Visualization, Automated Testing.

1. Introduction

Restaurants generate sales information through daily transactions involving different menu items and food categories. Analysing these records can help identify revenue-generating items, understand customer purchasing patterns, and observe changes in sales over time.

However, raw sales records are not always easy to interpret directly. Data may require validation and cleaning before meaningful calculations can be performed. Presenting the results in tables and charts can make the information easier to understand.

This project implements a basic restaurant sales analysis system using Python. It processes structured CSV data, calculates revenue-related metrics, and generates charts that summarize the results.

The current implementation is a command-line application that produces chart image files. It is a foundation for a future interactive dashboard rather than a fully interactive web application.

2. Problem Statement

Restaurant sales records contain useful information about menu-item performance and revenue, but manually calculating and comparing these values can be time-consuming and prone to errors.

The problem addressed by this project is to develop a simple, reproducible Python application that can load restaurant sales data, validate and clean records, calculate useful sales metrics, and generate visual representations of the results.

3. Objectives

The main objectives are:

1. Load restaurant sales data from a CSV file.
2. Validate the dataset's required columns.
3. Remove duplicate records and rows containing missing or invalid values.
4. Calculate revenue for each sales record.
5. Calculate total revenue across all records.
6. Analyse revenue by menu item and food category.
7. Calculate the quantity sold for each menu item.
8. Calculate revenue by date.
9. Generate charts using Matplotlib.
10. Verify important calculations and cleaning operations using automated tests.

4. Scope of the Project

The project currently supports:

- CSV-based sales data input.
- Basic data validation and cleaning.
- Revenue calculation using quantity and unit price.
- Grouped sales analysis by item, category, and date.
- Terminal-based reporting.

- Static chart generation.
- Automated testing.
- Source-code organization into separate modules.

The current scope does not include a database, user authentication, live restaurant data, online deployment, or interactive dashboard controls.

5. Tools and Technologies

Technology	Purpose
Python	Main programming language
pandas	Data loading, cleaning, grouping, and analysis
Matplotlib	Generating data visualizations
pytest	Automated testing
CSV	Input data storage format
Visual Studio Code	Development environment
Git	Version control
GitHub	Source-code hosting and collaboration
GitHub Actions	Planned automated testing workflow

The GitHub Actions workflow has been created in the project, but its successful execution on GitHub must be verified after the repository is pushed.

6. Dataset Description

The input dataset is stored at `data/restaurant_sales.csv`.

It contains six demonstration records with the following five columns:

Column	Description
Date	Date of the recorded sale
Item	Name of the menu item
Category	Food category
Quantity	Number of portions sold
Price	Price per portion in Indian rupees

Revenue is calculated using the following formula:

Revenue = Quantity × Price

For example, selling 10 portions of Paneer Tikka at ₹220 per portion produces revenue of ₹2,200.

The dataset is a small sample created for educational and demonstration purposes. It does not represent actual restaurant transaction records.

7. Methodology

The project follows a sequential data-processing workflow.

7.1 Data Loading

The `src/data_loader.py` module loads the CSV file using pandas. The dataset is stored in a DataFrame for subsequent processing.

7.2 Data Cleaning

The `src/data_cleaning.py` module performs the following operations:

- Checks whether all required columns are present.
- Removes exact duplicate records.
- Removes rows with missing values in required columns.
- Converts dates into datetime values.
- Converts quantity and price into numeric values.
- Removes rows containing invalid converted values.
- Retains records with positive quantities and non-negative prices.

The cleaning function returns a cleaned copy of the dataset.

7.3 Sales Analysis

The `src/sales_analysis.py` module contains functions for:

- Calculating record-level revenue.
- Calculating total revenue.
- Calculating revenue by menu item.
- Calculating quantity sold by menu item.
- Calculating revenue by food category.
- Calculating revenue by date.

Grouped results are sorted where appropriate to make comparisons easier.

7.4 Data Visualization

The `src/visualization.py` module uses Matplotlib to generate two chart images:

1. `outputs/revenue_by_item.png` — a bar chart comparing revenue across menu items.
2. `outputs/daily_revenue.png` — a line chart showing revenue by date.

The charts are saved to the output directory and can be opened separately from the command-line application.

7.5 Main Program

The `main.py` file coordinates the workflow. It loads the dataset, cleans the records, calculates sales metrics, prints results in the terminal, and generates the charts.

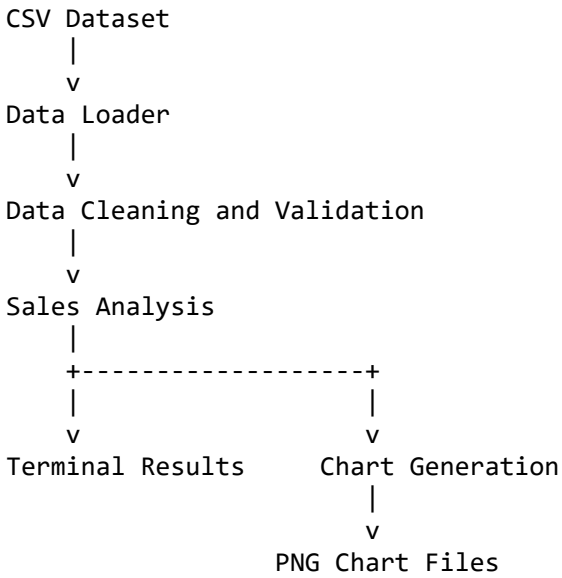
7.6 Automated Testing

The project uses pytest to check the correctness of important functions.

The test suite includes checks for revenue calculation, total revenue, revenue ranking by item, quantity ranking by item, duplicate removal, invalid-row removal, and handling missing required columns.

8. System Architecture

The application follows a modular structure:



Separating the application into modules improves readability, simplifies testing, and makes future modifications easier.

9. Implementation Details

The project separates data loading, cleaning, analysis, and visualization into independent Python modules.

The main revenue calculation uses the quantity and price columns:

```
Revenue = Quantity * Price
```

The total revenue function sums the calculated revenue values. Grouping operations calculate revenue by item, category, and date. A separate grouping operation calculates total quantity sold by menu item.

The visualization module creates a bar chart and a line chart and saves them as PNG files. The main program provides a single entry point for executing the workflow.

The project also includes a requirements file, test documentation, a data dictionary, a README, and a GitHub Actions workflow configuration.

10. Results and Discussion

The application was executed against the included six-record dataset.

10.1 Dataset Processing

Measure	Result
Records loaded	6
Records remaining after cleaning	6
Required data columns	5
Total revenue	₹15,040

All six sample records remain after cleaning because the included dataset contains no duplicate records, missing required values, or invalid dates or numeric values.

10.2 Revenue by Menu Item

Menu Item	Revenue
Paneer Tikka	₹4,840
Dal Makhani	₹4,200
Gulab Jamun	₹3,600
Veg Biryani	₹2,400
Total	**₹15,040**

Paneer Tikka generates the highest revenue in the sample dataset.

10.3 Quantity Sold

Menu Item	Quantity Sold
Gulab Jamun	45
Paneer Tikka	22
Dal Makhani	15
Veg Biryani	10
Total	**92**

Gulab Jamun has the highest quantity sold, illustrating that the most frequently sold item is not necessarily the item generating the highest revenue.

10.4 Revenue by Category

Category	Revenue
Main Course	₹6,600
Starter	₹4,840
Dessert	₹3,600
Total	**₹15,040**

Main Course generates the highest revenue among the three categories in the sample data.

10.5 Daily Revenue

Date	Revenue
1 October 2026	₹10,300
2 October 2026	₹4,740
Total	**₹15,040**

The sample shows lower revenue on the second date than on the first. Because the dataset covers only two dates, this observation should not be interpreted as a long-term sales trend.

10.6 Generated Visualizations

The application generates the following output files:

- `outputs/revenue_by_item.png`
- `outputs/daily_revenue.png`

These visualizations provide a graphical summary of the revenue calculations.

Insert the actual generated chart images into this report when preparing the final submission.

11. Testing and Validation

The project includes seven automated tests.

Test Area	Expected Behavior
Record-level revenue	Revenue equals quantity multiplied by price
Total revenue	Total equals the sum of record-level revenue
Revenue by item	Items are grouped and ranked by revenue
Quantity by item	Items are grouped and ranked by quantity
Duplicate removal	Exact duplicate rows are removed
Invalid-row removal	Rows with invalid dates or non-positive quantities are removed
Missing columns	A clear `ValueError` is raised

The test suite was reported as passing during development. The application was also run successfully, and both chart files were verified to exist.

These results establish that the tested functions work for the included scenarios. They do not prove that every possible input or edge case has been tested.

12. Limitations

The current implementation has several limitations:

1. The dataset contains only six sample records.
2. The dataset is not sourced from a real restaurant.
3. The application is command-line based rather than an interactive web dashboard.
4. The visualizations are static PNG files.
5. The system does not use a database or external data source.
6. No predictive model or forecasting functionality has been implemented.
7. The current tests cover selected behaviors rather than all possible cases.
8. The workflow's successful execution on GitHub remains to be confirmed after publication.

These limitations define the current project's scope and provide opportunities for further development.

13. Future Enhancements

Possible improvements include:

- Developing an interactive interface using Streamlit.
- Adding a larger, documented restaurant sales dataset.
- Introducing date-range and category filters.
- Adding key performance indicators and interactive charts.
- Supporting database integration.
- Improving test coverage for additional invalid inputs and edge cases.
- Publishing the application online.
- Adding screenshots and verified GitHub Actions results to the documentation.

These are proposed improvements and are not part of the current implementation.

14. Conclusion

The Restaurant Sales Dashboard demonstrates how Python can be used to transform structured sales records into useful analytical results. The project implements CSV loading, data cleaning, revenue calculation, grouped sales analysis, chart generation, and automated testing in a modular application.

For the included demonstration dataset, the system calculates total revenue of ₹15,040, identifies Paneer Tikka as the highest-revenue item, identifies Gulab Jamun as the highest-quantity item, and determines Main Course as the highest-revenue category.

The project provides a foundation for further work in data analytics and visualization. Its current limitations, particularly the small sample dataset and absence of an interactive interface, should be considered when interpreting its results.

References

1. Python Software Foundation. *Python Documentation*. https://docs.python.org/3/
2. pandas Development Team. *pandas Documentation*. https://pandas.pydata.org/docs/
3. Matplotlib Development Team. *Matplotlib Documentation*. https://matplotlib.org/stable/

4. pytest Development Team. *pytest Documentation*. https://docs.pytest.org/
5. GitHub. *GitHub Actions Documentation*. https://docs.github.com/en/actions

---

Finalization note: Complete the student and institution details, insert the generated charts, verify the test results and GitHub Actions status, and review the report against the final project before submitting it.